

DeepSleep

08

:

15

:

55

:

41

天 时 分 秒

项目内容:

- 本项目要求实现一个完整的「类 ChatGPT / DeepSeek」的大模型对话系统
- 系统需支持用户通过 Web 页面与大语言模型（LLM）进行多轮对话，并具备基础的对话管理能力与上下文理解能力
- 项目需覆盖完整链路，包括：前端交互界面 + 后端服务 + 模型调用 + 数据持久化
- 系统应具备基本产品形态，能够支持真实用户使用场景，包括连续对话、历史记录管理以及良好的交互体验

项目结构:

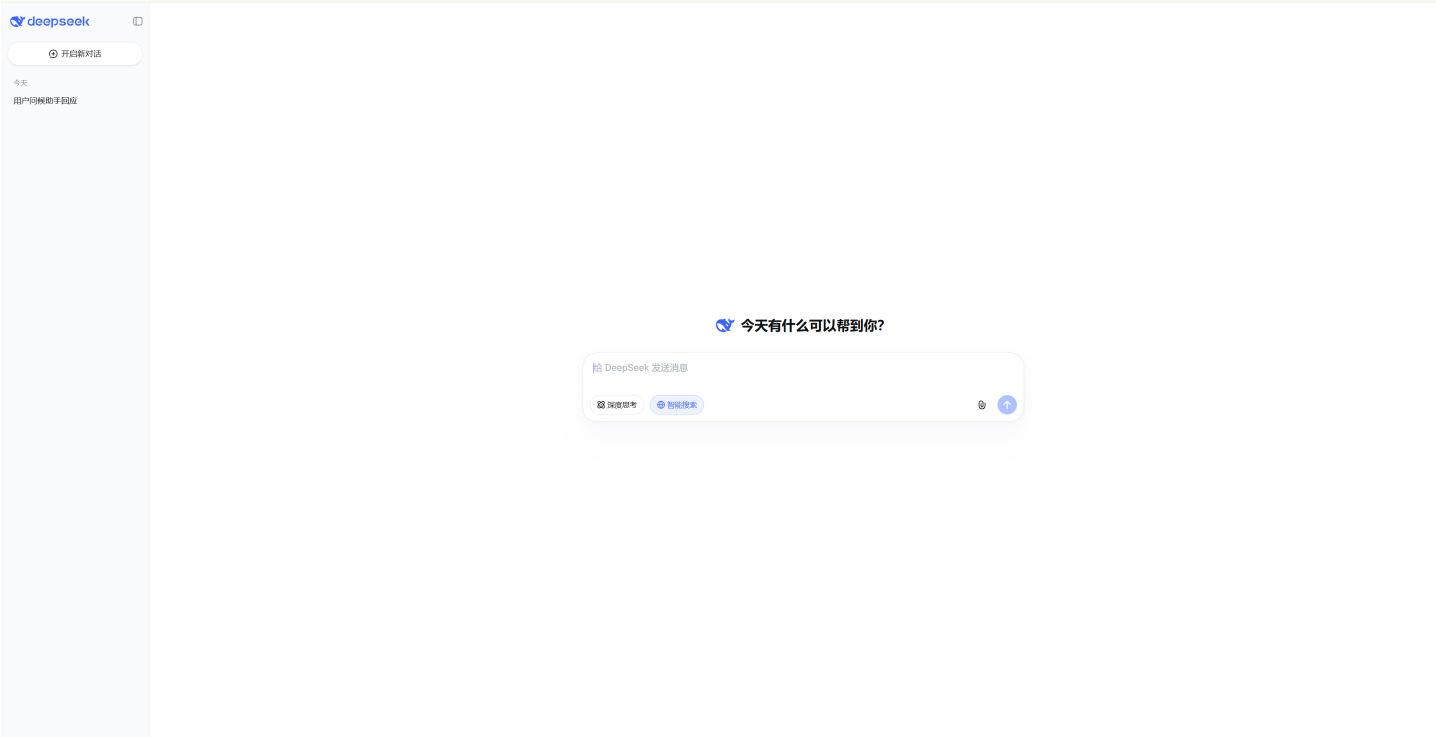
代码块

```
1  project-root/
2  |— frontend/           # 前端项目 (Vibe Coding)
3  |— backend/           # 后端项目
4  |— docs/              # 项目文档
5  |   |— api            # 接口文档
6  |   |   # api.md
7  |   |   #- 接口需遵循基本 RESTful 风格
8  |   |   #- 返回数据结构统一 (如 code / message / data)
9  |   |   # 至少需包含:
10 |   |   # - 登录接口
11 |   |   # - 会话创建接口
12 |   |   # - 会话列表接口
13 |   |   # - 消息发送接口 (流式说明)
14 |   |   # - 历史消息获取接口
15 |   |— sql             # 数据库建表sql
16 |   |   |— schema.sql  # 建表 SQL
17 |   |   |— init.sql    # 初始化数据 (可选)
```

```
18 | | └─ video/ # 演示视频
19 | |   └─ demo.mp4
20 | | └─ docker-compose.yml # 容器编排（如果有中间件）
21 | | └─ README.md # 说明文档
22 | | # - 项目介绍
23 | | # - 技术栈说明
24 | | # - 系统架构说明（可选）
25 | | # - 启动方式（前端 / 后端 / 数据库）
26 | | # - 环境依赖说明
27 └─ | # - 目录结构说明
```

UI:

仿照下面deepseek的页面:



功能要求:

基础功能

- i. 用户登录功能 (2分)
 - 支持用户注册 / 登录（账号密码即可）
 - 后续所有会话需与用户绑定（实现多用户隔离）
- ii. 与 LLM 进行会话（支持流式传输） (2分)
 - 用户输入问题后，后端调用大模型接口

- 支持流式返回（Streaming），逐字/逐段输出内容
- 前端需实时渲染回复过程（而非一次性展示）

iii. 会话支持上下文与历史记忆 (2分)

- 模型能够基于前文进行连续对话（多轮对话）
- 支持基础上下文裁剪（避免超长输入）

iv. 会话管理（新建会话 / 历史会话管理） (2分)

- 支持创建新会话（清空上下文）
- 支持展示历史会话列表（按时间或标题）

v. 点击历史会话可查看历史内容 (2分)

- 点击某个会话可加载该会话全部消息
- 支持继续在该会话中进行对话

vi. 使用 VibeCoding 完成前端开发 (2分)

- 使用 VibeCoding 工具生成 / 辅助生成前端页面
- UI 风格参考 DeepSeek / ChatGPT

加分项：

i. 联网搜索开关 (1分)

- 提供开关控制是否启用“联网搜索”

ii. 上下文工程优化 (0-6分)

- 提高回答质量或降低 token 消耗

iii. 模型选择功能 (1分)

- 支持在多个模型之间切换（如 GPT / DeepSeek / 本地模型）

iv. 图片 & 文件解析识别 (1-4分)

- 支持上传图片或文件（pdf / txt 等）

v. 构建 ReAct 模式开关（支持深度搜索） (2-6分)

- 实现简单 Agent 机制（思考 + 行动）
- 多轮推理（Thought → Action → Observation）
- 支持复杂问题分步解决

vi. 回答引用信息展示 (2分)

- 针对上下文所引用的信息进行回答

实现标准:

- 系统可正常启动（前端 + 后端 + 数据库）
- 用户可完成注册与登录流程
- 能够正常进行多轮对话
- 流式输出功能真实生效（非伪流式）
- 会话数据可持久化并正确加载
- 会话切换无数据错乱
- 项目具备完整 README 与启动说明

说明:

- 若核心功能无法运行，则对应分数不得分
- 加分项按实现完整度与效果评估

技术约束:

- 必须包含前后端完整架构（不可纯前端实现）
- 必须使用真实大模型 API（禁止 mock）
- 必须使用数据库进行数据持久化（禁止仅使用内存 / localStorage）
- 必须实现真实流式返回（SSE / WebSocket），禁止伪流式

项目要求:



- 后端功能实现要求80%手动实现
- 主动收集,了解相关知识
- 需要的技术栈可以自由拓展
- 尽量不依靠AI (我们拿到项目工程会审查代码)
- 在截止日期结束前提交

截止时间:

 下周四 23:59

提交方式:

- 将项目文件(仅包含上面项目结构即可)压缩为**.zip**格式, 飞书发送给  乔正阳